ADVANCED WEB TECHNOLOGIES

DETAILED PRACTICAL NOTES & VIVA QUESTION BANK

Expanded revision edition
Aligned to Assignments 01–04: Advanced JavaScript, React Basics, React Essentials and Advanced React.

This edition explains what each concept means, why it is used, how to write it, common mistakes, and how it appears in a small practical question.

Important: Your assignments specify broad application scenarios. The practical exam may ask for a small demonstration of one concept from those scenarios; focus on understanding the building blocks rather than memorising full applications.

Contents and study route

• Part A — JavaScript foundations: variables, conditions, loops, functions, arrays, objects and ES6.

• Part B — JavaScript assignment concepts: array methods, OOP, inheritance, DOM, browser storage, closures and asynchronous code.

• Part C — React: project setup, JSX, components, props, lists, state, lifecycle, Hooks and forms.

• Part D — Advanced React: Router, Redux Toolkit, Error Boundaries and JWT.

• Part E — practical recipes, debugging checklist and viva Q&A.;

How to use these notes tonight

Priority	Study topics	What you should be able to do
Very high	Array methods, OOP, DOM, localStorage	Write short JS programs without copying.
Very high	JSX, props, useState, useEffect, forms	Build a small React component and explain it.
High	Promises, async/await, Fetch API, closures	Explain asynchronous flow and write a safe example.
High	Router, Redux, Error Boundary, JWT	Explain terms and identify the correct code pattern.
Quick revision	Terminal commands, HTTP codes, viva	Answer definitions and troubleshoot setup.

Practical answering structure: state the data → write the function/component → connect the event/action → show the output → test one edge case.

1. JavaScript execution and basic syntax

What JavaScript does

JavaScript is a programming language used to make web pages interactive. HTML defines structure, CSS defines presentation, and JavaScript defines behaviour. In a browser, JavaScript can access the DOM, respond to events, validate forms, store browser data and call APIs. Node.js runs JavaScript outside the browser.

Variables: var, let and const

```
var oldCount = 1;     // function-scoped; can be redeclared
let count = 2;         // block-scoped; can be reassigned
const appName = "AWT"; // block-scoped; cannot be reassigned

count = 3;             // valid
// appName = "React";  // TypeError: const binding cannot change
```

Scope: scope determines where a variable can be accessed. let and const are block-scoped (inside { }). var is function-scoped and can cause unexpected behaviour when used inside blocks.

const nuance: const prevents assigning a new value to the variable, but it does not freeze an array or object.

```
const student = { name: "Asha", marks: 75 };
student.marks = 80;       // allowed: object is being modified
// student = {};          // not allowed: reassignment
```

Data types

Type	Example	Notes
String	'Krishna', "React"	Text; use quotes or template literals.
Number	10, 3.14	Integers and floating-point values.
Boolean	true / false	Logical values.
Undefined	let x;	Declared but not assigned.
Null	let x = null;	Intentional absence of value.
Object	{ name: 'Asha' }	Key-value structure; arrays and functions are objects in JS.
BigInt	123n	Large integers.
Symbol	Symbol('id')	Unique identifiers.

Operators and conditions

```
const marks = 68;
if (marks >= 75) {
  console.log("Distinction");
} else if (marks >= 40) {
  console.log("Pass");
} else {
  console.log("Fail");
}

const result = marks >= 40 ? "Passed" : "Failed"; // ternary
```

• **Arithmetic:** +, -, *, /, %, **.

• **Comparison:** ===, !==, >, <, >=, <=. Prefer strict equality.

• **Logical:** && (AND), || (OR), ! (NOT).

• **Assignment:** =, +=, -=, *=.

• Use if/else for branching, switch for multiple fixed cases, and loops for repetition.

Loops

```
const marksList = [55, 82, 39];

for (let i = 0; i < marksList.length; i++) {
  console.log(marksList[i]);
}

for (const mark of marksList) {
  console.log(mark); // value of each item
}

let i = 0;
while (i < 3) {
  console.log(i);
  i++;
}
```

Common mistake: an infinite loop occurs if the loop condition never becomes false. Check that the counter or condition changes.

2. Functions, arrays, objects and ES6

Functions

```
// Function declaration
function calculateTotal(price, quantity) {
  return price * quantity;
}

// Function expression
const square = function (n) {
  return n * n;
};

// Arrow function
const double = n => n * 2;

console.log(calculateTotal(100, 3)); // 300
```

Parameters are variable names in a function definition. Arguments are the actual values passed when calling it. return sends a result back to the caller; console.log only displays a value.

Arrays

```
const fruits = ["apple", "banana"];
fruits.push("mango");       // add at end
fruits.pop();               // remove last
fruits.unshift("orange");  // add at start
fruits.shift();             // remove first
console.log(fruits.length);
console.log(fruits[0]);
```

An array is ordered and zero-indexed: the first element is at index 0. Arrays can contain values of different types, although application data is often kept consistent.

Objects

```
const employee = {
  id: 101,
  name: "Ravi",
  department: "IT",
  salary: 45000
};

console.log(employee.name);       // dot notation
console.log(employee["salary"]);  // bracket notation
employee.salary = 48000;          // update
employee.email = "ravi@example.com"; // add property
delete employee.email;            // remove property
```

ES6 features frequently used in assignments

```
// Template literal
const message = `Employee: ${employee.name}`;

// Destructuring
const { name, salary } = employee;

// Spread: copy/extend an array
const scores = [60, 70];
const allScores = [...scores, 80];

// Spread: copy/update an object
const updatedEmployee = { ...employee, salary: 50000 };

// Rest parameters
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}

// Default parameter
function welcome(user = "Guest") {
  return `Welcome ${user}`;
}
```

• Template literals use backticks and ${expression} for interpolation.

• Destructuring extracts properties from objects or elements from arrays.

• Spread expands an iterable/object; commonly used to copy arrays and objects.

• Rest collects remaining arguments into an array.

• Default parameters provide a value when an argument is omitted or undefined.

3. Array higher-order methods in depth

A higher-order function accepts another function as an argument or returns a function. Array methods such as map, filter, reduce, find and sort commonly receive callback functions.

Method	Use it when you need to…	Callback returns
forEach()	Perform an action for every element.	Return value ignored.
map()	Create a transformed array.	New value for each item.
filter()	Select items meeting a condition.	true or false.
reduce()	Combine all elements into one result.	Updated accumulator.
find()	Get the first matching item.	true when match found.
sort()	Reorder array elements.	Negative / zero / positive comparison.

Example data

```
const employees = [
  { id: 1, name: "Asha", salary: 45000, dept: "IT" },
  { id: 2, name: "Ravi", salary: 65000, dept: "HR" },
  { id: 3, name: "Neha", salary: 52000, dept: "IT" }
];
```

map(): transform every element

```
const names = employees.map(emp => emp.name);
// ["Asha", "Ravi", "Neha"]

const revised = employees.map(emp => ({
  ...emp,
  salary: emp.salary < 50000 ? emp.salary * 1.10 : emp.salary
}));
```

map() does not change the original array when used this way; it returns a new array. It is suitable for increasing salaries, extracting names or calculating a derived value for every record.

filter(): select matching records

```
const highEarners = employees.filter(emp => emp.salary > 60000);
const itEmployees = employees.filter(emp => emp.dept === "IT");
```

filter() returns a new array containing every element for which the callback returns true. If no element matches, the result is an empty array.

reduce(): calculate a summary

```
const payroll = employees.reduce((sum, emp) => {
  return sum + emp.salary;
}, 0);

const average = payroll / employees.length;
```

The first callback parameter is the accumulator (sum); the second is the current element. The initial value 0 is supplied after the callback. For an object accumulator, initialise with {} or an object containing the required fields.

find() and sort()

```
const ravi = employees.find(emp => emp.id === 2);
// first matching employee or undefined

const ascending = [...employees].sort((a, b) => a.salary - b.salary);
const descending = [...employees].sort((a, b) => b.salary - a.salary);
const alphabetical = [...employees].sort((a, b) =>
  a.name.localeCompare(b.name)
);
```

Sorting rule: if comparator returns a negative number, a comes before b; zero means equal; positive means a comes after b. For numeric fields, never rely on the default sort without a comparator.

Typical student result calculations

```
const students = [
  { name: "Asha", marks: 78 },
  { name: "Ravi", marks: 35 },
  { name: "Neha", marks: 91 }
];

const passed = students.filter(s => s.marks >= 40);
const topper = [...students].sort((a,b) => b.marks - a.marks)[0];
const totalMarks = students.reduce((sum,s) => sum + s.marks, 0);
const classAverage = totalMarks / students.length;
```

4. JavaScript OOP: classes, inheritance and encapsulation

Why OOP?

Object-oriented programming organises related data and behaviour into objects. It is useful when modelling real-world entities such as Employee, Patient, Book, Student or BankAccount.

Class and constructor

```
class Patient {
  constructor(id, name, age, disease) {
    this.id = id;
    this.name = name;
    this.age = age;
    this.disease = disease;
  }

  getDetails() {
    return `${this.name}, age ${this.age}, ${this.disease}`;
  }
}

const patient1 = new Patient(1, "Asha", 22, "Flu");
console.log(patient1.getDetails());
```

A class defines the structure and behaviour. The constructor runs when new creates an instance. this refers to the instance being initialised.

Getters and setters

```
class Employee {
  constructor(name, salary) {
    this.name = name;
    this._salary = salary;
  }

  get salary() {
    return this._salary;
  }

  set salary(value) {
    if (value >= 0) this._salary = value;
    else throw new Error("Salary cannot be negative");
  }
}

const e = new Employee("Neha", 50000);
console.log(e.salary); // getter
e.salary = 55000;      // setter
```

A getter is accessed like a property, not called like a normal method. A setter intercepts assignment and can validate input. The underscore is a naming convention, not true privacy.

Inheritance and super

```
class Person {
  constructor(name) {
    this.name = name;
  }
  introduce() {
    return `I am ${this.name}`;
  }
}

class Student extends Person {
  constructor(name, course) {
    super(name); // initialise parent part
    this.course = course;
  }
  showCourse() {
    return `${this.name} studies ${this.course}`;
  }
}

const s1 = new Student("Asha", "MCA");
```

• Inheritance allows a child class to reuse and extend a parent class.

• extends establishes the inheritance relationship.

• super() calls the parent constructor; in a derived constructor it must be called before using this.

• Method overriding occurs when a child class provides its own implementation of an inherited method.

• Static methods belong to the class itself and are called using the class name.

Encapsulation using a closure

```
function createBankAccount(initialBalance) {
  let balance = initialBalance; // not directly accessible outside

  return {
    deposit(amount) {
      if (amount <= 0) return "Invalid amount";
      balance += amount;
      return balance;
    },
    withdraw(amount) {
      if (amount <= 0 || amount > balance) return "Invalid withdrawal";
      balance -= amount;
      return balance;
    },
    getBalance() {
      return balance;
    }
  };
}

const account = createBankAccount(1000);
account.deposit(250);
console.log(account.getBalance()); // 1250
```

The variable balance is private to the closure. Outside code can interact only through the returned methods. This is a common way to demonstrate encapsulation in JavaScript.

5. DOM manipulation: from HTML to interactive UI

What is the DOM?

The Document Object Model is the browser's object representation of an HTML document. JavaScript can select DOM nodes, change their content and styles, create new elements, remove elements and respond to user events.

Example HTML

```
<input id="taskInput" type="text" placeholder="Enter a task">
<button id="addBtn">Add Task</button>
<ul id="taskList"></ul>
```

Select elements and attach an event

```
const input = document.getElementById("taskInput");
const addButton = document.querySelector("#addBtn");
const taskList = document.querySelector("#taskList");

addButton.addEventListener("click", function () {
  const taskText = input.value.trim();

  if (taskText === "") {
    alert("Please enter a task");
    return;
  }

  const li = document.createElement("li");
  li.textContent = taskText;

  const removeButton = document.createElement("button");
  removeButton.textContent = "Delete";
  removeButton.addEventListener("click", () => li.remove());

  li.appendChild(removeButton);
  taskList.appendChild(li);
  input.value = "";
});
```

Important DOM properties and methods

API	Meaning / use
getElementById('id')	Find an element by its id.
querySelector('selector')	Find the first element matching a CSS selector.
querySelectorAll('selector')	Find all matching elements; returns a NodeList.
textContent	Read or set text content.
innerHTML	Read or set HTML markup; avoid inserting untrusted user input.
value	Read the current value of form controls.
style	Change inline CSS styles.
classList.add/remove/toggle	Add, remove or toggle CSS classes.
addEventListener(type, fn)	Listen for an event such as click, input or submit.
createElement(tag)	Create a new element node.
appendChild(node)	Append a child node.
remove()	Remove the element from the document.

Events you should know

• click — user clicks an element.

• input — value of an input changes as the user types.

• change — a control's value is committed/changed.

• submit — form is submitted.

• keydown / keyup — keyboard key pressed/released.

• DOMContentLoaded — HTML document has been parsed.

preventDefault()

```
form.addEventListener("submit", event => {
  event.preventDefault(); // stop normal browser form submission
  // Validate fields and handle data using JavaScript
});
```

Security note: use textContent for plain user-provided text. Assigning untrusted input to innerHTML can create cross-site scripting (XSS) risks.

6. localStorage, JSON and browser persistence

Web Storage provides key-value storage in the browser. localStorage persists across browser restarts for the same origin. sessionStorage is generally cleared when the tab/session ends.

Store and retrieve an array

```
const expenses = [
  { id: 1, title: "Travel", amount: 50 },
  { id: 2, title: "Food", amount: 120 }
];

// Convert array to JSON string before saving
localStorage.setItem("expenses", JSON.stringify(expenses));

// Read string and convert back into an array
const raw = localStorage.getItem("expenses");
const savedExpenses = JSON.parse(raw || "[]");

console.log(savedExpenses);
```

CRUD operations

```
// Create / add
savedExpenses.push({ id: 3, title: "Books", amount: 200 });
localStorage.setItem("expenses", JSON.stringify(savedExpenses));

// Read
const current = JSON.parse(localStorage.getItem("expenses") || "[]");

// Update a record
const updated = current.map(item =>
  item.id === 1 ? { ...item, amount: 75 } : item
);
localStorage.setItem("expenses", JSON.stringify(updated));

// Delete a record
const remaining = current.filter(item => item.id !== 2);
localStorage.setItem("expenses", JSON.stringify(remaining));

// Remove storage
localStorage.removeItem("expenses");
localStorage.clear();
```

Method	Purpose
setItem(key, value)	Save a key and string value.
getItem(key)	Read a value; returns null if missing.
removeItem(key)	Delete one key.
clear()	Delete all keys for the current origin.
JSON.stringify(value)	Convert JS data to JSON text.
JSON.parse(text)	Convert JSON text to JS data.

Common errors: JSON.parse(null) is not a useful default for missing data; use getItem(key) || '[]'. JSON.parse throws if the stored text is malformed, so production code may wrap parsing in try/catch. localStorage stores strings only.

Assignment connections

• Task manager: store task text, completion status and IDs; re-render after add/edit/delete.

• Movie booking: store selected movie, showtime and seats; calculate ticket price from seat count.

• Expense tracker: store transaction records; calculate income/expense/balance with reduce(); filter by date/category.

• localStorage is browser-specific and not a substitute for a secure database or server-side persistence.

7. Asynchronous JavaScript and Fetch API

Synchronous and asynchronous execution

Synchronous code executes in order, one operation at a time. Asynchronous operations such as timers and network requests complete later. JavaScript uses the event loop and task queues to coordinate this work.

```
console.log("A");
setTimeout(() => console.log("B"), 0);
console.log("C");
// Typical output: A, C, B
```

Callback

```
function getUser(callback) {
  setTimeout(() => {
    callback({ id: 1, name: "Asha" });
  }, 500);
}

getUser(user => console.log(user.name));
```

A callback is passed to another function and invoked when a task or event reaches a particular point. Deeply nested callbacks can become difficult to read and handle.

Promise

```
const payment = new Promise((resolve, reject) => {
  const paid = true;
  if (paid) resolve("Payment completed");
  else reject(new Error("Payment failed"));
});

payment
  .then(message => console.log(message))
  .catch(error => console.error(error.message))
  .finally(() => console.log("Payment process finished"));
```

A Promise has three states: pending, fulfilled or rejected. resolve(value) fulfils it; reject(error) rejects it. then handles success, catch handles failure, and finally runs either way.

async and await

```
async function makePayment() {
  try {
    const message = await payment;
    console.log(message);
  } catch (error) {
    console.error(error.message);
  }
}
```

await pauses the execution of the surrounding async function until the Promise settles. It does not block the entire browser or JavaScript runtime.

Fetch API

```
async function loadUsers() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/users");

    if (!response.ok) {
      throw new Error(`Request failed: ${response.status}`);
    }

    const users = await response.json();
    console.log(users);
  } catch (error) {
    console.error("Could not load users:", error.message);
  }
}
```

• fetch(url) sends a network request and returns a Promise for a Response.

• response.ok is true for successful HTTP status codes (200–299).

• response.json() reads and parses the response body asynchronously.

• Use try/catch for network failures and errors thrown after checking response.ok.

• Do not expose private API keys in frontend source code; use a backend proxy or appropriate public-key restrictions.

GET and POST examples

```
// GET
const response = await fetch("/api/students");
const students = await response.json();
```

```
// POST
const created = await fetch("/api/students", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "Asha", marks: 82 })
});
```

GET retrieves data. POST sends data to create/process a resource. The server must validate incoming data; frontend validation alone is not enough.

8. React fundamentals and Vite setup

What React is

React is a JavaScript library for building user interfaces from reusable components. React updates the UI when component state or props change. It is commonly used for single-page applications.

Create and run a Vite React project

```
npm create vite@latest awt-practice -- --template react
cd awt-practice
npm install
npm run dev
```

Open the local development URL printed in the terminal. The entry point is usually src/main.jsx, which renders the root App component.

Typical structure

```
awt-practice/
  index.html
  package.json
  src/
    main.jsx
    App.jsx
    App.css
    components/
      Header.jsx
      BookCard.jsx
      EmployeeCard.jsx
```

JSX rules

• Return one parent element or a Fragment.

• Use className for CSS classes and htmlFor for label association.

• Close all JSX tags, including input and img.

• Use curly braces to embed JavaScript expressions.

• Use conditional expressions or logical AND to show content conditionally.

• Component names must begin with a capital letter.

```
function Greeting({ name }) {
  const hour = new Date().getHours();
  return (
    <section className="greeting">
      <h1>Hello, {name}</h1>
      <p>{hour < 12 ? "Good Morning" : "Good Evening"}</p>
    </section>
  );
}
```

Components, props and composition

```
function BookCard({ title, author, price }) {
  return (
    <article>
      <h3>{title}</h3>
      <p>By {author}</p>
      <p>■{price}</p>
    </article>
  );
}

function BookStore() {
  return (
    <main>
      <h1>Featured Books</h1>
      <BookCard title="Clean Code" author="Robert Martin" price={500} />
      <BookCard title="Atomic Habits" author="James Clear" price={350} />
    </main>
  );
}
```

Props are inputs passed from a parent to a child. They are read-only. Component composition means assembling a larger page from smaller components. For an Employee Directory, you might create EmployeeCard, DepartmentDetails, SearchPanel and NavigationMenu.

Lists and keys

```
const students = [
  { id: 1, name: "Asha" },
  { id: 2, name: "Ravi" }
];

function StudentList() {
```

```
  return (
    <ul>
      {students.map(student => (
        <li key={student.id}>{student.name}</li>
      ))}
    </ul>
  );
}
```

A key helps React match each rendered item with its previous version. Use a stable unique ID when possible. Avoid array index if items can be reordered, inserted or removed.

9. React state, Hooks and lifecycle

Props versus state

Props	State
Passed into a component by its parent.	Owned/managed by the component.
Read-only to the receiving component.	Updated using a setter or setState.
Used to configure a child component.	Used for data that changes over time.
Changes when parent provides new values.	A state update schedules a re-render.

useState

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Add</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

The first array item is the current state. The second is the setter. The initial value is 0. Calling setCount schedules a render with the new value.

Functional state updates

```
setCount(previousCount => previousCount + 1);
```

Use the functional form when the next state depends on the previous state. It avoids relying on a potentially stale value, especially when updates are queued.

Visitor counter example

```
function VisitorCounter() {
  const [entered, setEntered] = useState(0);
  const [exited, setExited] = useState(0);
  const current = entered - exited;

  function visitorEnters() {
    setEntered(prev => prev + 1);
  }

  function visitorExits() {
    if (current > 0) setExited(prev => prev + 1);
  }

  function reset() {
    setEntered(0);
    setExited(0);
  }

  return (
    <div>
      <p>Entered: {entered}</p>
      <p>Exited: {exited}</p>
      <h2>Inside: {current}</h2>
      <button onClick={visitorEnters}>Enter</button>
      <button onClick={visitorExits}>Exit</button>
      <button onClick={reset}>Reset</button>
    </div>
  );
}
```

useEffect

```
import { useEffect, useState } from "react";

function CounterMessage() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Count: ${count}`;
    console.log("Count changed:", count);
  }, [count]);

  return <button onClick={() => setCount(c => c + 1)}>{count}</button>;
}
```

Dependency array	Effect behaviour
Omitted	Runs after every render.
[]	Runs after initial mount; cleanup on unmount.
[count]	Runs after mount and when count changes.

The dependency list tells React which reactive values the effect uses. Missing dependencies can cause stale values. Effects are for synchronising with external systems, not for every calculation that can be done during render.

Cleanup example: timer

```
useEffect(() => {
  const timerId = setInterval(() => {
    setTime(new Date().toLocaleTimeString());
  }, 1000);

  return () => clearInterval(timerId);
}, []);
```

The returned function is cleanup. React runs it before re-running the effect and when the component unmounts.

Class lifecycle for the assignment

```
class Clock extends React.Component {
  state = { time: new Date().toLocaleTimeString() };

  componentDidMount() {
    this.timer = setInterval(() => {
      this.setState({ time: new Date().toLocaleTimeString() });
    }, 1000);
  }

  componentWillUnmount() {
    clearInterval(this.timer);
  }

  render() {
    return <h2>{this.state.time}</h2>;
  }
}
```

componentDidMount runs after the component is mounted. componentWillUnmount runs before it is removed. This class-based example is included because your assignment specifically asks for lifecycle methods; function components usually use Hooks today.

10. React forms and validation

Controlled components

A controlled input gets its current value from React state. Its onChange handler updates that state. This makes the component the source of truth and allows validation and conditional feedback.

```
import { useState } from "react";

function AdmissionForm() {
  const [form, setForm] = useState({
    name: "", email: "", mobile: "", gender: "",
    city: "", course: "", address: "", percentage: ""
  });
  const [errors, setErrors] = useState({});
  const [submitted, setSubmitted] = useState(false);

  function handleChange(event) {
    const { name, value } = event.target;
    setForm(prev => ({ ...prev, [name]: value }));
  }

  function validate() {
    const next = {};
    if (!form.name.trim()) next.name = "Name is required";
    if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(form.email))
      next.email = "Enter a valid email";
    if (!/^\d{10}$/.test(form.mobile))
      next.mobile = "Enter exactly 10 digits";
    const percentage = Number(form.percentage);
    if (form.percentage === "" || percentage < 0 || percentage > 100)
      next.percentage = "Enter a percentage from 0 to 100";
    return next;
  }

  function handleSubmit(event) {
    event.preventDefault();
    const validationErrors = validate();
    setErrors(validationErrors);
    if (Object.keys(validationErrors).length === 0) {
      setSubmitted(true);
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name} onChange={handleChange} />
      {errors.name && <p>{errors.name}</p>}

      <input name="email" value={form.email} onChange={handleChange} />
      {errors.email && <p>{errors.email}</p>}

      <input name="mobile" value={form.mobile} onChange={handleChange} />
      {errors.mobile && <p>{errors.mobile}</p>}

      <input name="percentage" value={form.percentage} onChange={handleChange} />
      {errors.percentage && <p>{errors.percentage}</p>}

      <button type="submit">Submit</button>
      <button type="button" onClick={() => {
        setForm({ name:"", email:"", mobile:"", gender:"",
          city:"", course:"", address:"", percentage:"" });
        setErrors({});
        setSubmitted(false);
      }}>Reset</button>
      {submitted && <p>Admission form submitted!</p>}
    </form>
  );
}
```

Input types to remember

Control	React pattern
Text/email/number	value={state} and onChange={handler}
Radio	checked={state === option} and onChange
Select	value={state} and onChange
Textarea	value={state} and onChange
Checkbox	checked={booleanState} and onChange

Validation checklist from the assignment

• Student name must not be empty; use trim() to reject spaces-only input.

• Email should match a reasonable email pattern or be validated by the backend.

• Mobile number must contain exactly 10 digits: /^\d{10}$/.

• Percentage must be numeric and within 0–100 inclusive.

• Display each message near its field and stop submission when errors exist.

• After successful validation, display the submitted student details.

• Reset should clear both form state and validation/submission messages.

Important: HTML input type=email/required can provide browser validation, but the assignment asks for React validation messages. Implement explicit checks in the submit handler.

11. React Router: student portal

What routing solves

A single-page application can show different views based on the URL without requesting an entirely new HTML page for each navigation. React Router maps URL paths to components.

Install

```
npm install react-router-dom
```

Basic routes, links, dynamic student ID and 404

```
import {
  BrowserRouter, Routes, Route, Link, useParams
} from "react-router-dom";

const students = [
  { id: "1", name: "Asha", course: "MCA" },
  { id: "2", name: "Ravi", course: "MCA" }
];

function StudentList() {
  return (
    <ul>
      {students.map(student => (
        <li key={student.id}>
          <Link to={`/students/${student.id}`}>{student.name}</Link>
        </li>
      ))}
    </ul>
  );
}

function StudentDetails() {
  const { id } = useParams();
  const student = students.find(item => item.id === id);

  if (!student) return <h2>Student not found</h2>;
  return <h2>{student.name} — {student.course}</h2>;
}

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> |
        <Link to="/students">Students</Link> |
        <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/" element={<h1>Home</h1>} />
        <Route path="/students" element={<StudentList />} />
        <Route path="/students/:id" element={<StudentDetails />} />
        <Route path="/about" element={<h1>About portal</h1>} />
        <Route path="*" element={<h1>404 - Page not found</h1>} />
      </Routes>
    </BrowserRouter>
  );
}
```

Component / Hook	Purpose
BrowserRouter	Enables routing using browser URL history.
Routes	Container for matching Route definitions.
Route	Maps a path to an element/component.
Link	Navigates without full page reload.
useParams()	Reads dynamic route parameters such as id.
*	Wildcard route, often used for a not-found page.

Route order: React Router v6+ ranks route matches, so place the most specific paths normally; use path="*" as the catch-all not-found route.

12. Redux Toolkit: expense manager

Why Redux?

React component state is suitable for local UI state. Redux is useful when multiple components need to read or update shared application state. The store provides a central place for that state and a predictable update flow.

Redux data flow

```
Component dispatches an action
          ↓
      Redux Store
          ↓
     Slice Reducer
          ↓
     New state stored
          ↓
Subscribed components re-render
```

Term	Definition
Store	Holds the application's Redux state.
Action	Describes an event or requested state change.
Payload	Data carried by an action.
Reducer	Calculates next state from current state and action.
Dispatch	Sends an action to the store.
Selector	Reads a part of state.
Slice	Redux Toolkit object combining a slice name, initial state, reducers and generated actions.

Install

```
npm install @reduxjs/toolkit react-redux
```

Create expense slice and store

```
// src/features/expenses/expenseSlice.js
import { createSlice } from "@reduxjs/toolkit";

const expenseSlice = createSlice({
  name: "expenses",
  initialState: [],
  reducers: {
    addExpense: (state, action) => {
      state.push(action.payload);
    },
    deleteExpense: (state, action) => {
      return state.filter(expense => expense.id !== action.payload);
    }
  }
});

export const { addExpense, deleteExpense } = expenseSlice.actions;
export default expenseSlice.reducer;
```

```
// src/store.js
import { configureStore } from "@reduxjs/toolkit";
import expenseReducer from "./features/expenses/expenseSlice";

const store = configureStore({
  reducer: {
    expenses: expenseReducer
  }
});

export default store;
```

Provide the store to React

```
// src/main.jsx
import React from "react";
import ReactDOM from "react-dom/client";
import { Provider } from "react-redux";
import store from "./store";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

Read state and dispatch actions

```
import { useDispatch, useSelector } from "react-redux";
import { addExpense, deleteExpense } from "./features/expenses/expenseSlice";

function ExpenseList() {
  const expenses = useSelector(state => state.expenses);
  const dispatch = useDispatch();

  const total = expenses.reduce((sum, item) => sum + Number(item.amount), 0);

  return (
    <div>
      <h2>Total: ■{total}</h2>
      {expenses.map(item => (
        <p key={item.id}>
          {item.description} — ■{item.amount}
          <button onClick={() => dispatch(deleteExpense(item.id))}>
            Delete
          </button>
        </p>
      ))}
      <button onClick={() => dispatch(addExpense({
        id: Date.now(), description: "Travel",
        category: "Transport", amount: 50
      }))}>Add sample expense</button>
    </div>
  );
}
```

Why does state.push() appear to mutate? Redux Toolkit uses Immer inside createSlice reducers. It converts the mutation-like code into immutable state updates. Do not mutate Redux state directly in ordinary components.

13. Error Boundaries and JWT authentication

Error Boundary

A React Error Boundary catches JavaScript errors during rendering and lifecycle methods in its descendant component tree. It can display fallback UI instead of allowing that part of the UI to crash.

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error) {
    return { hasError: true };
  }

  componentDidCatch(error, info) {
    console.error("Error Boundary caught:", error, info);
  }

  render() {
    if (this.state.hasError) {
      return <h2>Course details could not be displayed.</h2>;
    }
    return this.props.children;
  }
}

// Use it around a component that may fail:
<ErrorBoundary>
  <CourseDetails />
</ErrorBoundary>
```

Limitations: Error Boundaries do not catch errors in event handlers, asynchronous callbacks such as setTimeout, server-side rendering, or errors thrown by the boundary itself. Use try/catch and Promise error handling for those.

JWT concepts

JWT stands for JSON Web Token. It is a compact token format commonly used to carry claims between a client and server. A server can issue a signed token after verifying a user's credentials.

Part	Role
Header	Token type and signing algorithm metadata.
Payload	Claims, such as subject/user ID and expiry.
Signature	Allows the server to verify that the token was not altered.

The three parts are commonly represented as header.payload.signature, separated by dots. The payload is Base64URL-encoded, not encrypted by default. Never put passwords or secrets in it.

Authentication flow

• 1. User submits email/username and password to the login endpoint.

• 2. Backend validates credentials and creates a signed JWT.

• 3. Client receives the token and stores/manages it according to the security design.

• 4. Client sends the token with protected API requests using Authorization: Bearer .

• 5. Backend verifies signature, expiration and authorization before returning protected data.

• 6. Logout clears the client-side session/token and redirects to login.

```
const response = await fetch("/api/student/dashboard", {
  method: "GET",
  headers: {
    Authorization: `Bearer ${token}`
  }
});

if (response.status === 401) {
  // Token missing, invalid or expired: send user to login
}
```

Security distinction: hiding a Dashboard component in React is only a user-interface guard. The backend must validate the token on every protected endpoint. For browser applications, secure HttpOnly, SameSite cookies may be preferable to exposing tokens to JavaScript, depending on the architecture.

HTTP status codes

Code	Meaning	Typical use
200	OK	Successful GET or request.
201	Created	A resource was created.
400	Bad Request	Invalid input or request format.
401	Unauthorized	Authentication is missing or invalid.
403	Forbidden	User is authenticated but lacks permission.
404	Not Found	Resource or route does not exist.
500	Internal Server Error	Unexpected server-side failure.

14. Assignment-by-assignment practical preparation

Assignment 01 — Advanced JavaScript

Assignment task	Core implementation to practise
Student Management	Array of objects; add; find topper; filter marks >= 40; sort by marks.
Library Management	Book objects; search/filter by category; update; delete; sort by title/price.
Employee class	Constructor; salary getter/setter; map for increment; filter high earners.
Hospital records	Classes/inheritance; destructuring/spread; filter admitted patients; reduce bill revenue.
Attendance/payroll	Attendance objects; compute salary; generate individual salary summary.
Task manager	DOM input/button/list; add, edit, complete, delete; localStorage.
Movie booking	Movie/show/seat selection; calculate price; update booking summary; persist selection.
Expense tracker	Transaction objects; totals using reduce; search/filter; edit/delete; localStorage.
Weather API	Input city; fetch API; check response.ok; show temperature/condition; handle invalid city.
Currency converter	Select currencies; fetch rates; amount × rate; show update date; handle failures.
Shopping cart closure	Private items array; add/remove/get total methods.
Student result analysis	map/filter/reduce/find/sort; percentages, grades, average and toppers.
Banking simulator	Closures for balance; callbacks/Promises/async; validate deposit/withdraw/transfer.

Assignment 02 — React Basics

• Book Store: build reusable components for category, featured book, bestseller, review and banner.

• Employee Directory: represent employees as objects and render reusable EmployeeCard components.

• Use props to pass each book/employee record into its card component.

• Separate common navigation/header/search panels from repeated record cards.

Assignment 03 — React Essentials

• JSX: create a component, render a dynamic list with map(), and conditionally show a greeting.

• Class lifecycle: start a timer in componentDidMount and clear it in componentWillUnmount.

• Hooks: useState for entered/exited counts; useEffect for a message whenever count changes.

• Forms: controlled text/radio/select/textarea controls; validate; show errors; display submitted data; reset.

Assignment 04 — Advanced React

• Router: Home, Students, Student Details using /students/:id, About and 404.

• Redux: add/delete expense; display all expenses; calculate total; state belongs in Redux store.

• Error Boundary: wrap CourseDetails; catch render error; show friendly fallback; log diagnostic details.

• JWT: login, receive token, manage token, protect dashboard, attach Bearer token, logout and handle expiry.

15. Debugging and common practical mistakes

Symptom	Likely cause / correction
map is not a function	Variable is not an array; inspect data and initialise with [].
Cannot read properties of null	DOM selector found no element; check id, selector and script timing.
Input value is empty	Read input.value after the event; check correct selector and trim whitespace.
Nothing happens on click	Check event listener, button type, function name and console errors.
localStorage shows [object Object]	You stored an object directly; use JSON.stringify().
JSON.parse error	Stored string is invalid JSON; inspect key and handle parse errors.
fetch returns data but UI says error	Check response.ok and API response shape; status 404/500 may still resolve fetch.
React component not rendering	Check export/import, component capitalization, JSX syntax and root render.
React list warning	Add a stable unique key to the outer element returned by map().
State display does not update	Do not mutate state directly; call setter with a new value/object/array.
useEffect runs repeatedly	Check dependency array and whether effect updates one of its own dependencies.
Timer runs multiple times	Clear interval in cleanup; avoid starting timers during render.
Route shows blank page	Check BrowserRouter wrapper, Route path, element prop and import.
Redux state undefined	Check reducer key, Provider wrapper, store import and selector path.
401 from protected API	Token may be missing/expired/invalid; check Bearer header and server validation.

Before declaring the practical complete

- Does the app start without terminal errors?
- Does the required button/event actually work?
- Does the output match the question?
- Have you tested empty/invalid input?
- Are errors handled and messages understandable?
- Can you explain the key concept in two or three sentences?

16. Viva question bank — JavaScript

Q. What is JavaScript?
A. A high-level language used to implement interactive behaviour in web pages and other environments such as Node.js.

Q. How is JavaScript different from Java?
A. They are distinct languages with different runtimes, type systems and ecosystems; similar names do not mean they are the same.

Q. What is the difference between == and ===?
A. == allows type coercion; === compares without coercing types. Prefer === for predictable comparisons.

Q. What is hoisting?
A. JavaScript processes declarations before execution in their scope. var is initialised as undefined; let/const exist in a temporal dead zone until declaration.

Q. What is scope?
A. The region of code where a variable can be accessed: commonly global, function or block scope.

Q. What is an arrow function?
A. A shorter function syntax that also uses lexical this rather than defining its own this.

Q. What is a callback?
A. A function passed to another function to be invoked later or at a specific point.

Q. What is a higher-order function?
A. A function that accepts another function as an argument or returns a function.

Q. map() versus filter()?
A. map transforms every element; filter selects elements satisfying a condition.

Q. filter() versus find()?
A. filter returns all matches in an array; find returns the first match or undefined.

Q. What does reduce() do?
A. It accumulates array elements into one result, such as a total, average or grouped object.

Q. Why provide an initial value to reduce()?
A. It defines the starting accumulator and avoids special behaviour for empty arrays.

Q. Does sort() mutate?
A. Yes, sort() changes the original array. Spread-copy before sorting if the original must be preserved.

Q. What is an object?
A. A collection of properties represented as key-value pairs.

Q. What is destructuring?
A. Syntax for extracting values from arrays or properties from objects into variables.

Q. Spread versus rest?
A. Spread expands iterable/object values; rest collects remaining values into an array or object.

Q. What is a class?
A. A template for creating objects with initial data and methods.

Q. What is a constructor?
A. A special class method that initialises an instance when new is used.

Q. What is this?
A. A reference determined by the call context; in a method call it commonly refers to the object before the dot.

Q. What is inheritance?
A. A child class reuses and extends a parent class using extends.

Q. What does super() do?
A. Calls the parent constructor from a child constructor.

Q. What is encapsulation?
A. Bundling data and related methods while controlling direct access to internal state.

Q. What is a closure?
A. A function retaining access to variables in its lexical outer scope.

Q. What is the DOM?
A. The browser's object-based representation of an HTML document.

Q. What is event bubbling?
A. An event propagates from the target element upward through its ancestors.

Q. What does preventDefault() do?
A. Cancels the browser's default action for an event.

Q. innerHTML versus textContent?
A. innerHTML parses/sets markup; textContent treats content as text and is safer for untrusted plain text.

Q. What is localStorage?
A. Persistent browser storage for string key-value pairs, scoped to an origin.

Q. Why use JSON.stringify()?
A. To convert JavaScript objects/arrays into JSON text for storage or transmission.

Q. What is a Promise?
A. An object representing the eventual result of an asynchronous operation.

Q. What are Promise states?
A. Pending, fulfilled and rejected.

Q. What is async/await?
A. Syntax for working with Promises in a sequential-looking style; async functions return Promises.

Q. Does fetch reject on 404?
A. Usually no. Check response.ok or response.status and handle unsuccessful HTTP responses explicitly.

17. Viva question bank — React and advanced React

Q. What is React?
A. A JavaScript library for creating component-based user interfaces.

Q. What is a component?
A. A reusable unit of UI that can receive props and manage state.

Q. What is JSX?
A. An HTML-like syntax extension for describing UI inside JavaScript.

Q. Why component names begin with uppercase?
A. It lets JSX distinguish custom React components from built-in HTML tags.

Q. What are props?
A. Read-only values passed from a parent component to a child.

Q. What is state?
A. Data managed by a component that can change and cause a re-render.

Q. Props versus state?
A. Props are passed in; state is managed internally or through a state-management system.

Q. What is component composition?
A. Combining small reusable components to create larger interfaces.

Q. Why are keys needed in lists?
A. They help React identify which list items changed between renders.

Q. What is a Fragment?
A. A way to group adjacent JSX elements without adding an extra DOM node.

Q. What is useState?
A. A Hook that gives a function component state and a setter function.

Q. Why should state not be mutated directly?
A. React relies on state updates to determine when to render; direct mutation may not trigger a correct update.

Q. What is useEffect?
A. A Hook for synchronising a component with external systems such as timers, subscriptions or network requests.

Q. What does an empty dependency array mean?
A. The effect runs after mount; cleanup runs on unmount, subject to development Strict Mode behaviour.

Q. Why return cleanup from useEffect?
A. To remove subscriptions, timers or listeners and prevent leaks or duplicate work.

Q. What is a controlled input?
A. An input whose value is driven by React state and updated through onChange.

Q. What is a lifecycle method?
A. A class-component method associated with mounting, updating or unmounting.

Q. What does componentDidMount do?
A. Runs after a class component is mounted into the DOM.

Q. What is React Router?
A. A routing library that maps URL paths to UI components.

Q. What is useParams?
A. A Router Hook that reads dynamic route parameters.

Q. What is a 404 route?
A. A catch-all route that renders a not-found page for unmatched paths.

Q. What is Redux?
A. A predictable state-management library for shared application state.

Q. What is a Redux action?
A. An object describing an event or requested state change, often with a payload.

Q. What is a reducer?
A. A function that calculates the next state from the current state and an action.

Q. What is dispatch?
A. The function used to send an action to the Redux store.

Q. What is Redux Toolkit?
A. The recommended Redux toolset that simplifies store and reducer setup.

Q. What is createSlice?
A. A Redux Toolkit function that creates a slice reducer and corresponding action creators.

Q. What does useSelector do?
A. Reads selected state from the Redux store.

Q. What does useDispatch do?
A. Returns the dispatch function used to send Redux actions.

Q. What is an Error Boundary?
A. A class component that catches render/lifecycle errors in descendants and displays fallback UI.

Q. Can Error Boundaries catch errors in event handlers?
A. No. Handle those with try/catch or Promise error handling.

Q. What is JWT?
A. JSON Web Token, a compact signed token format often used for authentication claims.

Q. What are the three JWT parts?
A. Header, payload and signature.

Q. Is a JWT payload secret?
A. No. It is encoded, not encrypted by default. Do not store passwords or secrets in it.

Q. What is a Bearer token?
A. An access token sent in the HTTP Authorization header as Bearer followed by the token.

Q. Why is backend validation required?
A. A user can bypass client-side checks; the server must verify the token and permissions on protected endpoints.

Q. 401 versus 403?
A. 401 means authentication is missing/invalid; 403 means the request is understood but access is forbidden.

18. Final high-yield revision sheet

Memorise these patterns

```
// Filter passed students
const passed = students.filter(s => s.marks >= 40);

// Calculate total marks
const total = students.reduce((sum, s) => sum + s.marks, 0);

// Find topper
const topper = [...students].sort((a,b) => b.marks - a.marks)[0];

// Update one object in an array
const updated = employees.map(e =>
  e.id === targetId ? { ...e, salary: e.salary * 1.10 } : e
);

// localStorage
localStorage.setItem("data", JSON.stringify(data));
const data = JSON.parse(localStorage.getItem("data") || "[]");

// React state
const [count, setCount] = useState(0);
setCount(prev => prev + 1);

// Dynamic list
{items.map(item => <li key={item.id}>{item.name}</li>)}

// Router parameter
const { id } = useParams();

// Redux
const value = useSelector(state => state.someKey);
const dispatch = useDispatch();

// Fetch
const response = await fetch(url);
if (!response.ok) throw new Error("Request failed");
const data = await response.json();
```

Last 45 minutes before the exam

Time	Revision
15 min	JavaScript array methods, classes, inheritance, closures.
10 min	DOM selectors/events, localStorage and JSON.
10 min	React components, props, useState, useEffect and controlled forms.
5 min	Router, Redux, Error Boundaries and JWT.
5 min	Terminal commands and speak viva answers aloud.

How to answer viva: (1) define the term, (2) state why it is used, (3) give a tiny example. Keep the answer direct. If you are unsure, explain the part you understand instead of inventing details.

Source alignment: The topics and assignment scenarios in this guide are based on the four AWT assignments you shared. Explanations and compact code patterns expand those topics for revision; they are not a claim that every implementation detail was explicitly prescribed by the assignment sheet.